

AI Features, MCP Integration & Security Architecture

A technical account of the platform's generative-AI components, the Model Context Protocol tooling, the document-extraction "policy engine," and the security controls enforcing them

System	JSD Emergency Services — ambulance/fire dispatch platform
Backend	.NET 8 AWS Lambda (TransportApi, VoiceAgent) + DynamoDB + API Gateway HTTP API
AI runtime	Amazon Bedrock — Nova Lite (<code>eu.amazon.nova-lite-v1:0</code>)
Report basis	Direct source inspection — every claim below is cited to a file and line range

Contents

- 1. Scope and reading guide — what "AI" actually means in this system
- 2. VoiceAgent: Bedrock-backed conversational dispatch intake
- 3. Open Knowledge Format (OKF): the voice agent's data input mechanism
- 4. MCP integration: "jsd-cloudwatch"
- 5. The "policy engine": what is and isn't RAG
- 6. Security architecture across all of the above
- 7. Known gaps, config-dependent risks, and legacy/dead code
- 8. Summary matrix

1. Scope and reading guide

This report covers every AI/ML-adjacent component actually present in the codebase, plus the security controls that gate them. It draws a hard line between three categories that are easy to conflate:

Category	Meaning	Found here
Generative AI	An LLM (Amazon Bedrock Nova Lite) actually performs inference over supplied text/documents.	2 PLACES
Statistical / deterministic analytics	Aggregation, averages, percentiles, Little's Law, centroid geometry — no model inference.	2 ENDPOINTS
Not implemented	Described in design docs or named suggestively, but no working code exists.	RAG, ADMIN CHATBOT

Headline finding: there is no retrieval-augmented generation (RAG) pipeline anywhere in this codebase — no vector database, no embeddings API calls, no similarity search, no document chunking. Two components use the word "policy" and one uses "knowledge," and neither is RAG. Section 3 and Section 5 explain precisely what each one actually does instead, because the mechanisms are legitimate and worth documenting accurately — just not as RAG.

2. VoiceAgent: Bedrock-backed conversational dispatch intake

`lambda/VoiceAgent/Function.cs` (472 lines) is a .NET 8 Lambda that lets a caller describe an emergency in natural language and turns it into a structured, validated dispatch request. It is the only component in the system that performs live LLM inference at request time.

2.1 Architecture

Only Amazon Bedrock is used for generative AI — there is no Amazon Transcribe, Polly, or Lex integration anywhere in the repository. Speech capture/playback, where used, happens client-side via the browser's Web Speech API in `src/portal/VoiceAgent.jsx`; the Lambda only ever sees text.

```
Bedrock model: eu.amazon.nova-lite-v1:0 (env BEDROCK_MODEL_ID)
API:           Amazon.BedrockRuntime - Converse
Auth:          Cognito JWKS bearer token (conditional - see §6.4)
Downstream:    x-api-key call to TransportApi POST /emergencies
```

2.2 Request flow

Step	What happens	Reference
1	CORS preflight short-circuit; Bearer JWT verified against Cognito JWKS if enabled.	<code>Function.cs:81-95</code>
2	Request body capped to the last 20 conversation turns, each truncated to 1,000 characters before it ever reaches the model.	<code>Function.cs:100-110</code>
3	Reference locations fetched from TransportApi (<code>/reference/locations</code>) and cached per warm container.	<code>Function.cs:320-336</code>
4	First turn with no user input yet → a scripted greeting is returned; Bedrock is not called.	<code>Function.cs:121-122</code>
5	<code>ExtractSlots()</code> sends the transcript + system prompt (including the OKF bundle, §3) to Bedrock <code>Converse</code> ; the reply is regex-matched for a JSON object and parsed defensively.	<code>Function.cs:254-301</code>
6	Extracted fields are validated against <i>fixed, deterministic</i> enums — not the model's judgment.	<code>Function.cs:54-55</code>
7	Free-text location is resolved to a known location ID by fuzzy token-overlap scoring — plain string matching, not an LLM call.	<code>Function.cs:351-372</code>
8	Missing slots trigger scripted (non-generated) follow-up questions.	<code>Function.cs</code> (slot-prompt logic)
9	A confirmation summary is shown; dispatch only fires once the caller/UI sends <code>confirmed: true</code> .	<code>Function.cs:157-172</code>
10	<code>DoDispatch()</code> POSTs the structured booking to TransportApi's <code>/emergencies</code> using a static <code>x-api-key</code> , then converts the structured response into a human-readable reply.	<code>Function.cs:186-249</code>

Deterministic enums enforced regardless of model output

```
ValidSev    = ["Critical", "Urgent", "Normal"]  
ValidCases = ["Cardiac", "Trauma", "General", "Maternity", "Pediatric"]
```

This is the load-bearing safety property of the design: the LLM is used purely as a natural-language *slot extractor*. It never writes directly to the dispatch system, never picks a hospital, never decides severity outside these fixed values — every field it proposes is checked against a closed enum or a real reference-data ID before use. A hallucinated severity value ("Serious", say) is simply rejected and re-prompted, not passed through.

3. Open Knowledge Format (OKF): the voice agent's data input

The question of how the voice agent gets its domain knowledge has a concrete, already-implemented answer: a small bundle of Markdown documents, loaded once per cold start and injected into the Bedrock system prompt on every turn.

3.1 Mechanism

Element	Detail
Loader	<code>LoadOkf()</code> , <code>Function.cs:57-79</code>
Source directory	<code>infra/knowledge/</code> , bundled alongside the Lambda binary
Contents loaded	<code>emergency-types/index.md</code> , <code>vehicles/index.md</code> , and every file under <code>locations/</code> except its own <code>index.md</code>
Sub-documents present	<code>emergency-types/{cardiac,general,maternity,pediatric,trauma,index}.md</code> , <code>locations/*.md</code> , <code>vehicles/</code>
Injection point	Concatenated verbatim into the Bedrock system prompt on <i>every</i> call — <code>Function.cs:256-268</code>
Fallback	If the knowledge directory is missing at runtime, silently falls back to a plain <code>id = name</code> list pulled from the live reference API — <code>Function.cs:258</code>

What this is: static context injection ("prompt-stuffing") of a curated, version-controlled document set. It is simple, fully deterministic in what the model can see, cheap (no per-query retrieval cost), and auditable — you can read exactly what the model was told, every time, by reading the Markdown files in the repo.

What this is not: there is no chunking, no embeddings, no vector similarity search, and no dynamic selection of which documents to include based on the query. Every call gets the entire bundle, unconditionally. This works because the corpus is small (a handful of short Markdown files); it would not scale to a large or frequently-changing knowledge base without becoming a genuine RAG pipeline (see §5.3 for what that would require).

3.2 Why "OKF" rather than raw prompt text

Keeping the knowledge as separate Markdown files under version control — rather than inline strings in `Function.cs` — means domain experts can edit emergency-type definitions, vehicle capability notes, or location descriptions without touching Lambda code, and every change is visible in `git diff`. It is a content-authoring convenience, not a retrieval architecture.

4. MCP integration: "jsd-cloudwatch"

The **Model Context Protocol** server is a local, on-demand developer tool — not a persistent network service, and not exposed to end users of the emergency-dispatch application itself.

4.1 Architecture

Element	Detail
Server file	<code>infra/mcp/cloudwatch-server.mjs</code> (226 lines), Node.js
Registration	<code>.mcp.json</code> (git-ignored — never committed; contains local API key)
Transport	stdio JSON-RPC 2.0 — Claude Code spawns <code>node infra/mcp/cloudwatch-server.mjs</code> as a child process per request; no open port, no daemon
Backend it proxies	Single endpoint: <code>GET /infra/metrics</code> on the TransportApi Lambda

4.2 Tools exposed

Tool	Purpose
<code>get_lambda_metrics</code>	Invocations, error rate, avg/p99 duration, throttles, cold starts, time series
<code>get_recent_errors</code>	Recent ERROR/WARN CloudWatch log lines
<code>query_logs</code>	Keyword/substring filter over recent log lines
<code>get_infra_summary</code>	Derived healthy/degraded status against fixed thresholds (error rate ≥5%, any throttles, p99 >8000ms, cold starts >20)
<code>get_infra_metrics</code>	Raw payload — same data backing the Infra Health admin dashboard

4.3 Auth model

A static `x-api-key` header (`MCP_API_KEY`) is sent on every proxied call. Server-side, `Auth.cs` maps the key to the source label `MCP` via the `API_KEYS` env var, and the `Scopes` dictionary restricts that source to the `infra` resource only (`Auth.cs:42-52`) — the MCP key *cannot* be used to touch dispatch, fleet, or emergency data even if leaked, because the backend enforces the scope regardless of what the caller requests. No AWS IAM credentials ever leave the backend Lambda; CloudWatch/X-Ray calls happen server-side under the Lambda's own least-privilege role.

Flag: the local `.mcp.json` in this working copy holds a live-looking API key in plaintext. The file is correctly git-ignored (confirmed via `git check-ignore`) and is not in git history, so this is a local-machine exposure surface, not a committed secret — but it underscores that the key itself is static and long-lived rather than a rotating token, and should be treated with the same care as any other credential on disk.

4.4 What is proposed but not built

`docs/MCP-Security-And-Chatbot-Options.txt` describes a second option — an in-app admin chatbot at `POST /infra/chat`, embedded in the Infra Health dashboard, also backed by Bedrock Nova Lite. No such route exists in `Function.cs`, no `ChatPanel` component exists in the frontend, and `services/api.js` has no matching method. This is a documented proposal only.

5. The "policy engine": what is and isn't RAG

Two unrelated features share the word "policy." Conflating them is the single easiest mistake to make when describing this system, so they are treated separately here.

5.1 `infra/policy-agent` — Bedrock document-extraction, genuinely wired end-to-end

Element	Detail
Entry point	<code>infra/policy-agent/policy-sync.mjs</code> (115 lines), standalone Lambda
Trigger	<code>POST /policy</code> on TransportApi — admin-only, 5 MB cap, base64-decoded PDF, uploaded to S3, synchronously invokes <code>psiog-policy-sync</code> — <code>Function.cs:246-273</code>
Model call	Bedrock <code>Converse</code> with the raw PDF sent as a document content block (lines 79-89) — <i>one shot, no retrieval step, no chunking</i>
Extraction target	Exactly 6 named parameters: <code>speed_kmh</code> , <code>mass_patient_threshold</code> , <code>patients_per_ambulance</code> , <code>max_units</code> , <code>autocomplete_minutes</code> , <code>severity_order</code>
Guardrail	<code>sanitize()</code> (lines 42-53) clamps every value to hardcoded bounds — e.g. <code>speed_kmh</code> $\in [5, 120]$ — before anything is applied
Apply step	<code>applyToBackend()</code> (lines 91-95) writes the sanitized JSON to the TransportApi Lambda's <code>POLICY_CONFIG</code> environment variable via <code>UpdateFunctionConfigurationCommand</code>
Consumption	Read at TransportApi cold start (<code>Function.cs:43</code>); <code>speed_kmh</code> feeds ETA calculation (line 69); <code>patients_per_ambulance</code> / <code>max_units</code> / <code>mass_patient_threshold</code> feed mass-casualty dispatch sizing (lines 664-666)

The code's own comments are precise about the boundary: it controls *parameters*, and explicitly does not generate or replace code. This is document-to-structured-config extraction with a hard-bounded output space — a narrow, safe use of an LLM, not a general reasoning agent and not RAG (there is no corpus to retrieve from; the entire input is one PDF, sent whole, every time).

5.2 `PolicyControls.jsx` / `policyService.js` — an unrelated fuel/vehicle-grade config feature

`src/services/policyService.js` (27 lines) manages a completely different "policy" domain: fuel allowances and permitted vehicle grades per job level (`job_level` bands, `allowed_vehicle_types` , `monthly_fuel_cap_litres` , `shuttle_rides`). It is plain client-side getters over a config object populated from a REST call. **There is no AI, no Bedrock, and no relationship to the policy-agent above** beyond the shared English word. This distinction matters because "policy engine" language in casual conversation could plausibly refer to either — this report is explicit that only §5.1 involves generative AI.

5.3 On RAG specifically

A repository-wide search for retrieval/embeddings/vector-store patterns (Pinecone, OpenSearch, pgvector, FAISS, any Bedrock *embeddings* API call, any chunking utility) returned no matches. If a genuine RAG capability were added later — for example, to let the voice agent or an admin chatbot answer from a larger, frequently-updated corpus than the current small OKF bundle — it would need, at minimum: a chunking strategy, an embeddings model call (Bedrock supports Titan Embeddings for this), a vector index (OpenSearch Serverless with the k-NN plugin is the natural AWS-native choice here), and a query-time similarity-search step before the generation call. None of that exists today; the current OKF approach (§3) is a reasonable substitute at the current corpus size and should not be described as RAG in other materials.

6. Security architecture

6.1 JWT verification (Cognito JWKS)

Implemented identically in both Lambdas — `lambda/TransportApi/Auth.cs:68-100` and `lambda/VoiceAgent/Function.cs:377-405`. Fetches and caches (1-hour TTL, double-checked-lock via `SemaphoreSlim`) the Cognito user pool's JWKS, validates issuer, signature, token lifetime (5-second clock-skew tolerance), optional audience/ `client_id` match, and restricts `token_use` to `access` or `id`.

6.2 API key scoping (least privilege by caller identity)

`Auth.cs:31-52`: the `API_KEYS` env var maps each key to a source label — `CONSOLE`, `HOSPITAL`, `EDUCATION`, `DELIVERY`, `ADMIN`, `HR`, `FUEL`, `MCP` — and the `Scopes` dictionary restricts each label to specific POST resources. Only `CONSOLE` carries a wildcard. Enforced on every request (`Function.cs:108-109`), and the two analytics GET routes additionally require `admin || apiKeySource == "MCP"` (`Function.cs:897, 969`) — ordinary hospital/education/delivery/fuel keys cannot read analytics data at all.

6.3 DynamoDB synthetic/live data isolation

The live dispatch table (`TransportRequests`) and the synthetic historical table (`TransportRequestsHistorySynthetic`) are separate DynamoDB tables, never cross-referenced at request time. Both analytics endpoints read exclusively from the synthetic table and both responses carry an explicit machine-readable disclaimer: `"Based on synthetic historical data seeded for demonstration – not live dispatch records."` IAM reinforces the same boundary: `infra/sso-permission-set-policy.json` grants write access (`CreateTable/PutItem/BatchWriteItem`) only to the synthetic table's ARN — the live table is read-only in that policy.

6.4 CORS

Configurable via `ALLOWED_ORIGINS` (comma-separated allowlist) in both Lambdas; defaults to `*` if unset. The server echoes back the caller's origin only if it's on the allowlist. **This defaults open** — it is a deploy-time configuration responsibility, not a hardcoded restriction.

6.5 Input validation

Boundary	Control
VoiceAgent conversation turns	Capped to last 20 turns, 1,000 chars/turn
Policy PDF upload	5 MB cap, valid-base64 requirement
Extracted policy parameters	Hardcoded numeric clamps (e.g. <code>speed_kmh</code> $\in [5,120]$) applied before any value reaches production config
VoiceAgent slot values	Closed enums for severity/case type; fuzzy-match resolution (not LLM) for location IDs against real reference data

7. Known gaps, config-dependent risks, and legacy code

Auth is a runtime toggle, not a hard requirement

In both Lambdas, JWT enforcement only activates if `COGNITO_REGION` / `COGNITO_USER_POOL_ID` are set at deploy time. If those env vars are absent — a plausible misconfiguration, not a design intent — the endpoint falls through to API-key-only or fully open access. The JWKS signature verification itself appears sound; the risk is entirely in deploy-time configuration discipline, not a bypassable check.

Default-open CORS and plaintext API keys

`ALLOWED_ORIGINS` defaults to `*` unless explicitly locked down. `API_KEYS` is currently a plaintext Lambda environment variable by default — `infra/security/harden-aws.sh` offers a Secrets Manager migration, CSP/HSTS headers, a WAF rate-based rule, voice-endpoint throttling, and a cost budget, but every one of these is a *manually-run CloudShell subcommand*, not part of the automated `buildspec.yml` pipeline (which only builds the SPA, syncs to S3, and invalidates CloudFront). None of `harden-aws.sh`'s protections should be assumed "in place" without independently confirming they were actually run against the live environment.

Legacy/dead-code candidate: `backend/`

`backend/handler.mjs` (1,052 lines) + `backend/auth.mjs` is a full Node.js reimplementations of the same API surface as the current C# `lambda/TransportApi`, predating commit `52f0da2` ("Add C# .NET 8 Lambda rewrites... remove SES/SNS"). It is not referenced by `buildspec.yml`, and the OKF/policy-config features that exist today only exist in the C# code. It still ships with its own deploy script (`infra/deploy-backend.sh`) alongside the current one (`infra/deploy-csharp.sh`) — a strong candidate for removal, though this report cannot 100% confirm it is unreferenced in the live AWS account without checking deployed Lambda configuration directly.

8. Summary matrix

Component	Status	Notes
VoiceAgent (Bedrock Nova Lite slot extraction)	LIVE	LLM extracts structured fields; every field validated against closed enums / real reference data before use
OKF knowledge bundle	LIVE	Static Markdown prompt-stuffing, not retrieval — appropriate at current corpus size
MCP server (jsd-cloudwatch)	LIVE	Local studio dev tool, scoped API key, read-only observability
MCP-based admin chatbot (/infra/chat)	PROPOSED ONLY	Documented design, no code exists
Policy-agent (Bedrock PDF extraction)	LIVE	End-to-end: PDF → 6 clamped parameters → live dispatch config
PolicyControls (fuel config)	LIVE	Unrelated feature, zero AI involvement — do not conflate with policy-agent
Retrieval-augmented generation	NOT IMPLEMENTED	No vector store, embeddings, or retrieval step anywhere in the repo
Coverage-gap / AI Insights analytics	LIVE, STATISTICAL	Little's Law, centroid geometry, percentiles — no model inference; synthetic data only
Legacy Node.js backend (backend/)	LIKELY DEAD	Superseded by C# rewrite; not in the automated deploy pipeline
harden-aws.sh hardening suite	AVAILABLE, NOT AUTOMATED	Manual CloudShell subcommands; unclear if exercised against the live account